



Computer Organization & Assembly Language

Course: Computer Organization & Assembly Language (Lab) - CS-318

What are Registers?

- Registers are **tiny storage locations inside the CPU**.
- They are **very fast** compared to normal memory (RAM).
- The CPU uses them to **store data temporarily**, like numbers, addresses, or results of calculations.
- Think of registers like **small bowls on the CPU's table**, while RAM is like a kitchen shelf – far away and slower to reach.

8086 Internal registers 16 bits (2 bytes each)

AX	AH	AL	Accumulator
BX	BH	BL	Base
CX	CH	CL	Count
DX	DH	DL	Data

} Data group

AX, BX, CX and DX are two bytes wide and each byte can be accessed separately

SP	Stack pointer
BP	Base pointer
SI	Source index
DI	Destination index
IP	Instruction pointer

} Pointer and index group

These registers are used as memory *pointers*.

Flags _H	Flags _L	Status and control flags
--------------------	--------------------	--------------------------

6 status; 3 control ; 7 unused

ES	Extra
CS	Code
DS	Data
SS	Stack

} Segment group

Segment registers are used as base address for a segment

General Purpose/Data Registers

Following four registers are available to the programmer for general data manipulation:

- AX (Accumulator): Used in arithmetic, logic and data transfer instructions. Also required in multiplication, division and input/output operations.
- BX (Base): It can hold a memory address that points to a variable.
- CX (Counter): Act as a counter for repeating or looping instructions. These instructions automatically repeat and decrement CX and quit when equals to 0.
- DX (Data): It has a special role in multiply and divide operations. Also used in input/output operations.

Segment Registers

Store addresses of instruction and data in memory.

These values are used by the processor to access memory locations.

- CS (Code): Defines the starting address of the section of memory holding code.
- DS (Data): Defines the section of memory that holds most of the data used by programs.
- ES (Extra): This is an additional data segment that is used by some of the string instructions.
- SS (Stack): It defines the area of the memory used for stack

Pointers and Index Registers

These can be accessed only as 16 bit registers.

IP - instruction pointer: Always points to next instruction to be executed. IP register always works together with CS segment register and it points to currently executing instruction.

SI - source index register: Can be used for pointer addressing of data. Offset address relative to DS

DI - destination index register: Can be used for pointer addressing of data . Offset address relative to ES

- SI and DI used in string movement instructions.
- SP and BP are used to access data inside the stack segment
- BP - base pointer: Primarily used to access parameters passed via the stack. Offset address relative to SS
- SP – stack pointer: Always points to top item on the stack. Offset address relative to SS

Basic Assembly Program Structure

An assembly program usually has **3 main sections**:

Data Section (section .data / .data)

Used to store **constants or fixed data** (things that don't change).

Example: messages, file names, buffer sizes.

```
section .data
```

```
msg db 'Hello', 0Ah
```

BSS Section (section .bss / .bss)

Used to declare **variables** (memory reserved, but not initialized yet).

Example:

```
section .bss
```

```
number resb 1 ; reserve 1 byte
```

Text Section (section .text / .code)

Contains the actual **instructions/code** of the program.

Program execution starts here.

```
section .text
```

```
global _start ; Linux
```

```
_start:
```

```
    ; instructions go here
```

Comments

Comments start with a **semicolon** ;.

They are ignored by the assembler.

```
MOV AX, BX ; Copy BX into AX
```

Statements in Assembly

Three types:

- **Instructions** → Actual CPU commands (e.g., MOV, ADD, JMP).
- **Directives (pseudo-ops)** → Instructions for the assembler, not the CPU (e.g., .MODEL, .DATA).
- **Macros** → Shortcut blocks of code (like functions).

Directives (Special Instructions for Assembler)

- Tell the assembler how to arrange code and data.
- Do **not** generate machine instructions.

Examples:

- **.MODEL** → Choose memory model (tiny, small, etc.).
- **.STACK** → Set stack size.
- **.DATA** → Define initialized variables.
- **.CODE** → Begin code section.

- **END** → Marks end of program.
- **DOSSEG** → Align segments properly for DOS.

Memory Models (MASM, old DOS)

This decides how big your program can be and how code/data are stored:

- **TINY** → Code + data fit in 64KB (makes .COM files).
- **SMALL** → One code segment + one data segment (each <64KB).
- **COMPACT** → Code is small (<64KB), data can be bigger.
- **MEDIUM** → Data is small, code can be bigger.
- **LARGE** → Both code and data can be big, but arrays ≤ 64KB.
- **HUGE** → Same as LARGE but arrays can be bigger than 64KB.

Simple rule:

- Small programs (like Hello World) → **TINY / SMALL model**.
- Big programs with lots of data/code → **LARGE / HUGE model**.

Stack Segment (.STACK)

Defines memory reserved for stack (temporary storage for calls, variables).

Example:

```
.STACK 100h ; 256 words reserved for stack
```

Data Segment (.DATA)

Stores variables/constants that are initialized.

Example:

```
.DATA
```

```
num DB 10
```

Code Segment (.CODE)

Where the actual instructions go.

Example:

```
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
    ; your program here
    MOV AH, 4Ch
    INT 21h    ; exit to DOS
MAIN ENDP
END MAIN
```

Basic Assembly Command

1. **MOV** (Move / Copy)

```
MOV AX, 10    ; put 10 in AX
```

```
MOV BX, AX    ; copy value of AX into BX
```

2. **ADD** (Addition)

```
ADD AX, 5     ; AX = AX + 5
```

3. **SUB** (Subtraction)

```
SUB BX, 2     ; BX = BX - 2
```

4. **MUL / IMUL** (Multiply)

Unsigned uses MUL, signed uses IMUL.

```
MOV AX, 4
```

```
MOV BX, 3
```

```
MUL BX        ; DX:AX = AX * BX → AX = 12 (since small)
```

5. **DIV / IDIV** (Divide)

Unsigned uses DIV, signed uses IDIV.

```
MOV AX, 10
```

```
MOV BL, 2
```

```
DIV BL      ; AX = 10 / 2 → result = 5 (quotient in AL, remainder in AH)
```

6. **INC / DEC** (Increment / Decrement)

```
INC AX      ; AX = AX + 1
```

```
DEC BX      ; BX = BX - 1
```

7. **CMP** (Compare)

```
CMP AX, BX   ; sets flags (like AX - BX) but doesn't store result
```

8. **INT** (Interrupt – DOS services)

```
MOV AH, 09h
```

```
MOV DX, OFFSET Msg
```

```
INT 21h      ; print string
```

```
Msg DB 'Hello, DOS World!$'
```

- In **DOS real mode**, registers are **AX, BX, CX, DX** (16-bit), and most I/O is done through INT 21h.
- For .COM programs, usually ORG 100h is at the start, since DOS loads them at offset 100h.

Lab Exercise

Q1: Print Your Name

Write a program to display your full name on the screen using INT 21h.

Q2: Compare Two Numbers

Compare two numbers stored in registers.

If equal → print "EQUAL"

Otherwise → print "NOT EQUAL"

Q3: Print A to Z

Write a program to print all uppercase alphabets A–Z using a loop.

Q4: Print a Message 5 Times

Write a program that prints the message "HELLO" five times using a loop.

Q5: Print 0–9 (Digits)

Write a program to display all digits from 0 to 9 in one line.

Q6: Reverse Characters Using Stack

Push characters 'A', 'B', 'C' onto the stack, then pop them and display them back.

Expected output: CBA

Q7: Print Even Letters Only

Write a program to print only even-position alphabets:

Output: BDFHJ...Z

Q8: Jump Example

Write a program that checks a number in AX.

If AX contains 1, print "ONE".

Otherwise, print "NOT ONE".

Q9: Print Your Roll Number

Store your roll number in a message and print it using INT 21h.

Q10: Loop with Delay

Write a program that prints * * * * * with a small pause (using a loop as delay counter).
